

Title: Per-Layer and Per-Token Adaptive Quantization for Efficient LLM Serving

Group Members: Ernesto Ibanez Jr, Ushnesha Daripa, and Aditya Pise

Emails: eibanezj@asu.edu, udaripa@asu.edu, apise12@asu.edu,

Group leader: Ernesto Ibanez Jr

Background and Motivation

LLMs increasingly power applications such as question answering, coding assistants, document analysis, and enterprise AI systems. However, serving these models efficiently remains challenging on resource-limited hardware. During inference, two major consumers of GPU memory are model weights and the key-value (KV) cache maintained for active requests.

Production systems such as [vLLM](#) improve GPU memory efficiency through continuous batching and PagedAttention, but when large models (e.g., Llama-3) run on single commodity GPU, the memory needed for model weights and the growing KV-cache becomes the main bottleneck limiting throughput and usable context length.

To reduce this memory cost, current systems such as [TensorRT-LLM](#) rely on static quantization, using methods like [AWQ](#) and [GPTQ](#) to compress weights to one fixed low-bit format (e.g., INT8 or INT4) across the entire model. This static approach has two limitations. First, outlier sensitivity, which is when a few unusually large activation values force the quantization scale to stretch, reducing precision for the far more common normal values. Second, spatiotemporal blindness, which is when every layer and every token is compressed by the same fixed amount, even though some layers and tokens matter far more than others.

Addressing these limitations would enable high-fidelity LLM serving on cheaper hardware, without the accuracy loss static quantization currently forces. Recent work supports this direction. For example, [CALM](#) and [RAMP](#) show that adjusting precision per layer recovers accuracy lost to static quantization, and [Don't Waste Bits!](#) shows the same holds per token, using lightweight signals to adjust KV-cache precision during decoding. This motivates our per-layer and per-token adaptive quantization approach.

Research Problem Statement

LLM serving faces two memory pressures, fixed-size model weights, and a KV-cache that grows continuously during decoding. Current systems address both with static quantization which is simple but ignores the precision needs that vary across the model. We will study whether dynamically allocating precision, guided by measured importance rather than a fixed rule, can recover accuracy that static approaches sacrifice, without adding enough execution overhead to erase quantization's memory savings.

Workload: Inference serving for an open-weight LLM (e.g., Llama-3-8B) under a fixed GPU memory budget, covering both weights and the growing KV-cache.

Behavior we aim to improve: The memory-versus-accuracy tradeoff. Specifically, whether adaptive precision (per-layer for weights, per-token for the KV-cache) beats any single static precision choice.

Constraints: A fixed memory budget, a bound tolerance for accuracy loss, and the execution-efficiency limits discussed in our Technical Challenges section.

Objective: Determine an appropriate solution that dynamically minimizes memory footprint with acceptable model accuracy, while keeping latency competitive with static baselines.

Research Questions:

1. Can per-layer, sensitivity-guided precision allocation reduce weight memory footprint beyond static methods (AWQ, GPTQ) at comparable accuracy?
2. Can per-token, attention-guided KV-cache precision allocation reduce cache memory or extend usable context length beyond static KV-cache quantization, without meaningful accuracy loss or added latency?
3. Can both mechanisms run efficiently enough (avoiding dequantization overhead, warp divergence, and memory misalignment) that combined throughput stays competitive with static baselines?

Technical Challenges

Our research runs into multiple technical challenges, including the following:

1. **Dequantization Execution Overhead:** GPU Tensor Cores expect all values in one fixed format at a time, so tokens or layers stored at different precisions must first be unpacked and rescaled on general-purpose CUDA cores before any math happens. A simple fix would be converting everything back to one high-precision format first, but this would remove the memory savings that we got from quantization, since the same amount of data still has to move through memory. [MoBiQuant](#) reduces this cost by reconstructing higher precision on the fly from smaller stored values, but does not eliminate the overhead.
2. **Warp-Level Inefficiency:** GPUs run threads in fixed groups of 32 called warps, and they run fastest when every thread in a warp is doing the same thing. If threads are handling tokens at different precisions, the GPU either processes them one precision at a time (which is slow) or must constantly resort tokens into matching groups, which is really costly since token importance changes throughout decoding, requiring resorting at every step. [MoBiQuant's](#) token-aware router attempts this resorting, but doing it efficiently under constant change remains unsolved and is a key motivation for our kernel design.
3. **Estimation Importance Cheaply and Accurately:** The most accurate way to identify which layers or tokens can tolerate lower precision uses second-order sensitivity information (e.g., Hessian-based methods), which is too expensive to compute at LLM scale. Cheaper signals (entropy, gradient size, attention weight) are fast but not always trustworthy. [TASA](#) shows cheap signals can misidentify which layers actually matter most, silently hurting the model's quality. This tradeoff between "cheap" and "correct" is central to our project, since both our per-layer and per-token methods depend on a signal that is both fast enough to run and accurate enough to trust.

Initial Ideas

Idea 1: Measuring exactly how much precision a layer or token can lose usually requires expensive second-order methods like [HAWQ](#), which does not scale to billion-parameter models. Instead, we use two cheap proxies. For weights, we estimate each layer's sensitivity from how its output shifts under a small perturbation, computed in one pass over a small calibration set. For the KV-cache, we track each token's running average attention weight, a value the model already computes, at no added cost. This addresses the importance-estimation challenge, and differs from both full Hessian methods

(which are too expensive) and static baselines (which have no importance signals at all). [CALM](#), [RAMP](#), and “[Don’t Waste Bits!](#)” already show lightweight signals recover most of the accuracy of expensive ones, at both the layer and token level.

Idea 2: Letting every layer or token take an arbitrary precision would be inefficient to run. Threads doing different work can’t execute together, each precision needs separate unpacking, and the data no longer fits GPU memory alignment. We considered this fully flexible design and rejected it for these reasons. Instead, we use a small fixed set of levels (e.g., FP16, INT8, INT4), grouping all layers or tokens at the same level so they run together, implemented as a custom Triton kernel extending an existing FlashAttention-style kernel. This is the middle ground between static baselines and a fully flexible design, and it mirrors what related work already does, for instance, “[Don’t Waste Bits!](#)” and [KIVI](#) both restrict themselves to a small number of hardware-friendly groupings rather than arbitrary ones.

Evaluation Plan

Platform: ASU Research Computing’s Sol supercomputer, using its 80GB NVIDIA A100 GPUs. Our software stack includes PyTorch and Hugging Face Transformers for model handling, AutoGPTQ/AutoAWQ for reproducing static baselines, and vLLM/TensorRT-LLM for serving-level benchmarking. Our adaptive KV-cache mechanism will be implemented as a custom Triton kernel (CUDA if needed).

Models: Our primary model is Llama-3-8B-Instruct, the standard scale used across our cited literature (AWQ, GPTQ, KIVI, CALM), enabling direct comparison to published baselines numbers. Time permitting, we will also test Mistral-7B to show our method generalizes across architectures.

Datasets and Workloads: We will use WikiText-2 for perplexity (the standard across our cited literature) and HellaSwag for downstream accuracy, matching “[Don’t Waste Bits!](#)” for direct comparison. Since short contexts don’t meaningfully stress KV-cache growth, we will also use LongBench to evaluate accuracy and memory as context length increases.

Resource Feasibility: Llama-3-8B needs about 16GB for FP16 weights. Its KV-cache is lean (grouped-query attention, ~128KB/token, under 1GB at 4k tokens), so the full model comfortably fits on a single 40GB A100 which is well within Sol’s 80GB A100s. No training memory is needed, since our experiments are inference only. If GPU access is limited, our fallback is a smaller model (Llama-3.2-3B-Instruct or SmolLM-1.7B, also used in “[Don’t Waste Bits!](#)”), using the same methodology.

Baselines: We compare against (1) static FP16 as an accuracy upper bound, (2) static AWQ/GPTQ at INT8/INT4 (via vLLM/TensorRT-LLM) for weights, and (3) static KV-cache quantization for the cache. These test whether adaptive allocation actually improves on current standard practice.

Metrics: Perplexity and HellaSwag accuracy test whether adaptive precision preserves quality better than static baselines at matched memory. GPU memory consumption quantifies footprint reduction at matched accuracy. TTFT, TPOT, latency, and throughput verify that mixed-precision overhead doesn’t erase adaptivity’s efficiency gains.

Timeline and Workload Division

Period	Milestone	Member(s)
Weeks 4-6	Complete core literature review, set up Sol GPU access and model, reproduce static baselines	Ernesto, Ushnesha, Aditya
Weeks 7-8	Design per-layer importance signal, design per-token attention-tracking signal	Ernesto, Ushnesha
Weeks 9-11	Develop benchmark and conduct initial per-layer and per-token experiments	Ernesto, Ushnesha, Aditya
Weeks 12-13	Improve systems and complete experiments	Ernesto, Ushnesha, Aditya
Week 14	Analyze results and prepare presentation	Ernesto, Ushnesha, Aditya
Week 15-16	Final experiments and report	Ernesto, Ushnesha, Aditya

Meeting Plan

The team will meet weekly on Fridays at 4:00pm. Each member would share their progress, any issues they are having, and what their next steps are. Between meetings, tasks and progress will be tracked asynchronously on a shared Kanban board, giving each other continuous visibility into each member's status. All code, experiment scripts, and results will be maintained in a shared GitHub repository, organized by subsystem (weight quantization, KV-cache quantization, benchmarking) to support independent development before integration. The team plans to meet with the instructor approximately every two weeks, or more frequently around key milestones.

Risk Mitigation

Risk: The adaptive precision-allocation mechanism introduces enough dequantization, warp-divergence, or memory alignment overhead that end-to-end throughput falls below static quantization baselines, even if accuracy improves.

Mitigation: Fall back to a coarser adaptivity granularity, for example, per-chunk rather than per-token precision assignment. This would be trading some flexibility for hardware-friendly alignment, following the same design tradeoff [KIVI](#) makes between per-channel and per-token KV-cache quantization.

Risk: The selected model does not fit within available GPU memory on Sol, or GPU access is more limited than expected

Mitigation: Fall back to a smaller open-weight model (e.g., a 1-3B parameter model instead of Llama-3-8B).

References

- AWQ — Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., & Han, S. (2023). *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration*. arXiv:2306.00978.
- CALM — Zhang, Y., Zhang, X., Chen, W., Sun, L., & Yan, Y. (2025). *CALM: A CKA-Guided Adaptive Layer-Wise Modularization Framework for LLM Quantization*. arXiv:2512.16282.
- "Don't Waste Bits!" — Boroujeni, S. P. H., Mehrabi, N., Woods, P., Hillesheim, G., & Razi, A. (2026). *Don't Waste Bits! Adaptive KV-Cache Quantization for Lightweight On-Device LLMs*. arXiv:2604.04722.
- GPTQ — Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. arXiv:2210.17323.
- HAWQ — Dong, Z., Yao, Z., Gholami, A., Mahoney, M. W., & Keutzer, K. (2019). *HAWQ: Hessian AWARE Quantization of Neural Networks with Mixed-Precision*. arXiv:1905.03696.
- KIVI — Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., & Hu, X. (2024). *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache*. arXiv:2402.02750.
- MoBiQuant — Wang, D., Kim, J., Han, S., Gudovskiy, D., Nakata, Y., Okuno, T., Peong, K.-T., Jeon, K. E., Ko, J. H., Chen, Y., & Yang, H. (2026). *MoBiQuant: Mixture-of-Bits Quantization for Token-Adaptive Any-Precision LLM*. arXiv:2602.20191.
- PagedAttention / vLLM — Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention*. Proceedings of SOSP 2023.
- RAMP — Gautam, A. S., & Jha, S. (2026). *RAMP: Reinforcement Adaptive Mixed-Precision Quantization for Efficient On-Device LLM Inference*. arXiv:2603.17891.
- TASA — Wang, F., Xue, C., Liu, T., Shen, L., Liu, Y., & Ding, C. (2026). *Beyond Activation Alignment: The Alignment-Diversity Tradeoff in Task-Aware LLM Quantization*. arXiv:2607.00908.
- TensorRT-LLM — NVIDIA. (2024). *Optimizing LLMs for Performance and Accuracy with Post-Training Quantization*. NVIDIA Technical Blog.